Design Patterns II

THE VISITOR PATTERN

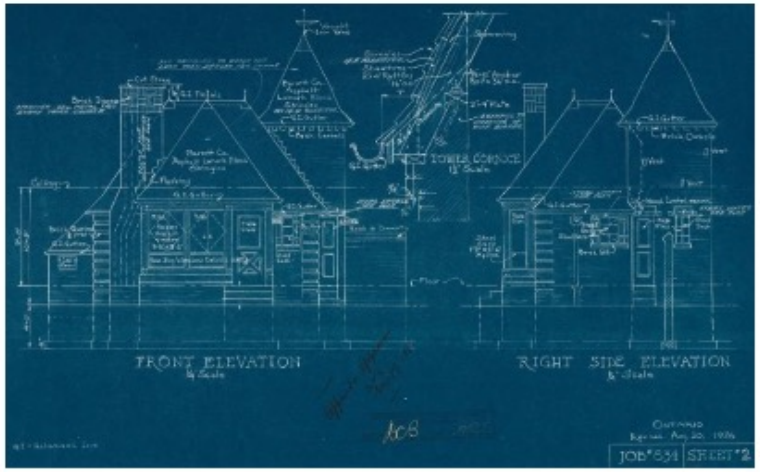
Design Pattern

A blueprint or schematic to solve a re-occurring problem.

- i.e. a common battle-tested solution to a frequently occurring problem.

Each pattern has a **name**, an **intent**, and **solves a specific problem**.

- We can use design patterns to communicate our thoughts!
- Unfortunately this is not always an exact science.



Design Patterns

Creational Patterns:

- Abstract Factory
- **Builder**
- Factory Method
- Singleton

Structural Patterns:

- Adapter
- Composite
- **Decorator**
- Proxy

Behavioral Patterns:

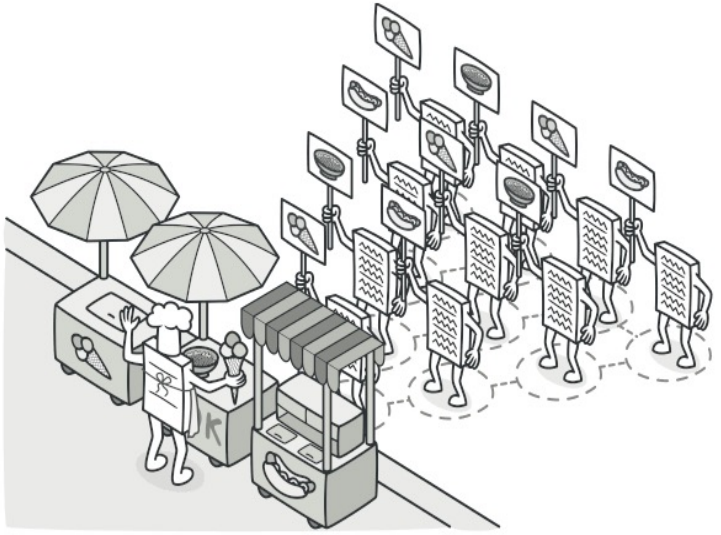
- Command
- Iterator
- Memento
- Null Object
- **Observer**
- State
- **Strategy**
- Visitor

And many many more…

The Visitor Pattern

- **Purpose**: Define new functionalities without introducing the modifications to an existing class.

Visitor is a behavioral design pattern that lets you separate algorithms from the objects on which they operate.



- Usually we start from a **composite class**:
  - List, Trees, Graphs, etc. of different elemts.

- Implement **new functionalites** by transvering elements.
  - No coded can be added to your composite class.

- **Visitor Pattern:**
  - Implement different ways of visiting the elements of the composite class
  - No code added to your composite class.